

Here's a list of **Generative AI (GenAI)** interview questions with **detailed answers**, tailored for roles involving work with LLMs, diffusion models, prompt engineering, RAG, and applied GenAI systems. These are categorized for clarity:

Core GenAI Concepts

1. What is Generative AI? How is it different from traditional AI?

Answer:

Generative AI refers to models that can generate new content (text, images, audio, etc.) rather than just making predictions or classifications.

Traditional AI focuses on discriminative tasks (e.g., spam detection), whereas GenAI creates novel data (e.g., ChatGPT generating code or images).

2. What are some common types of generative models?

Answer:

- **Autoregressive models** (e.g., GPT): predict next token given previous ones.
- **Variational Autoencoders (VAEs)**: encode and decode input with a latent space.
- **Generative Adversarial Networks (GANs)**: consist of a generator and discriminator.
- **Diffusion Models** (e.g., Stable Diffusion): gradually denoise data from noise.
- **Flow-based models**: invertible networks like RealNVP for exact likelihoods.

LLM-Specific Questions

3. Explain how transformer architecture powers LLMs.

Answer:

Transformers use self-attention to weigh the importance of tokens in a sequence. This enables parallel processing and long-range dependency modeling. Components include:

- **Multi-head self-attention**
- **Feed-forward layers**
- **Layer normalization**
- **Positional encoding**

4. What is the difference between GPT and BERT?

Answer:

Feature	GPT	BERT
Type	Autoregressive	Autoencoder
Training	Unidirectional (left-to-right)	Bidirectional
Task	Text generation	Classification, QA
Use case	Chatbots, code generation	Search, embeddings

5. What is prompt engineering?

Answer:

Prompt engineering is crafting input text to elicit desired outputs from LLMs. It includes:

- Few-shot or zero-shot prompting
- Instruction tuning
- Use of role-based prompting (e.g., "Act as a data scientist...")

Retrieval-Augmented Generation (RAG)

6. What is RAG and why is it useful?

Answer:

RAG combines retrieval from external knowledge bases (like vector databases) with generation. It improves factuality and enables real-time knowledge access for LLMs, which are otherwise static after training.

7. What components make up a RAG pipeline?

Answer:

- **Retriever:** fetches relevant documents (e.g., using FAISS or OpenSearch)
- **Embedder:** converts text into vector embeddings (e.g., SentenceTransformers)
- **Generator:** LLM that consumes retrieved documents to generate response
- **Post-processing:** for ranking, reranking, or summarizing

8. What challenges are associated with RAG systems?

Answer:

- **Latency** due to retrieval step
- **Context length limits** in LLMs
- **Inaccurate or hallucinated generations** if retrieval is poor
- **Embedding drift** when updating vector databases

Training & Fine-Tuning

9. What is LoRA and why is it used?

Answer:

LoRA (Low-Rank Adaptation) fine-tunes large models by injecting trainable low-rank matrices into specific layers. It reduces:

- Memory usage
- Training time
- Number of parameters updated

10. How do instruction tuning and RLHF differ?

Answer:

- **Instruction tuning:** supervised fine-tuning on prompt-response pairs.
- **RLHF** (Reinforcement Learning with Human Feedback): trains a reward model using human preferences and fine-tunes with Proximal Policy Optimization (PPO).

Multimodal & Diffusion Models

11. How do diffusion models work?

Answer:

Diffusion models add noise to data and learn to reverse this process. During inference, they start from noise and denoise step-by-step to produce realistic samples.

12. Compare GANs and diffusion models.

Answer:

Feature	GANs	Diffusion
Training	Adversarial	Denoising-based
Stability	Hard to train	More stable
Output quality	High (sometimes unstable)	High (more consistent)
Diversity	Can mode-collapse	Typically more diverse

Engineering & Deployment

13. How do you handle hallucinations in LLMs?

Answer:

- Add **retrieval** (RAG) to ground responses in facts

- Use **prompt engineering** to be more specific
- Apply **fact-checking** post-generation
- Fine-tune models on **domain-specific data**

14. What are strategies to reduce inference latency in GenAI systems?

Answer:

- Use quantization (e.g., INT8, GPTQ)
- Implement batching and caching
- Use smaller models (e.g., DistilGPT)
- Use hardware acceleration (e.g., GPUs, TPUs)
- Employ LoRA-adapted models for efficiency

15. How do you ensure safety and compliance in GenAI applications?

Answer:

- Use **moderation layers** (e.g., filters or classifiers)
- Apply **red-teaming** to test system robustness
- Maintain **audit logs**
- Use **fine-tuning** to align with safety guidelines
- Add **disclaimers** for generative content

 Bonus: Applied Scenarios

16. You built a chatbot using GPT-4 and it gives outdated answers. How would you fix it?

Answer:

- Add a **retriever** using RAG
- Use tools like **LangChain** or **LlamaIndex**

- Refresh your vector database periodically
- Optionally, fine-tune a smaller model on recent data

17. What is an embedding and how is it used in GenAI?

Answer:

An embedding is a dense vector representation of data (e.g., text) in high-dimensional space. In GenAI:

- Used in **retrieval**
- Power **similarity search**
- Enable **semantic clustering**

Would you like a downloadable version (e.g., PDF/CSV), or an expanded list by topic (e.g., RAG-only, Prompt Engineering-only)?

You said:

What is hallucination

ChatGPT said:

What is Hallucination in Generative AI?

Hallucination refers to when a generative AI model, especially a large language model (LLM), produces output that is **factually incorrect**, **fabricated**, or **inconsistent with reality**, but still **sounds plausible** or **confident**.

Example

Prompt: "Who is the current CEO of Twitter?"

Hallucinated Output: "Barack Obama is the current CEO of Twitter."

- This is a **hallucination** because the answer is **factually wrong**, even though the sentence is grammatically and stylistically correct.

Why Do Hallucinations Happen?

1. **Lack of grounding:** LLMs are trained on fixed data and don't have real-time access to the internet or external databases unless integrated with RAG.
2. **Statistical guessing:** Models predict the next word based on probabilities, not truth.
3. **Insufficient or biased training data:** If the model wasn't trained on enough correct or current information.
4. **Prompt ambiguity:** Vague or underspecified prompts may lead the model to "fill in the gaps" with guesses.

How to Reduce Hallucinations

1. **RAG (Retrieval-Augmented Generation):** Fetch relevant documents and ground the model's response.
2. **Prompt engineering:** Make prompts specific and well-scoped.
3. **Post-generation verification:** Use fact-checking APIs or models.
4. **Fine-tuning on domain-specific or curated factual data.**
5. **Smaller context window:** Trimming irrelevant data can reduce confusion.

Let me know if you want hallucination examples in **code generation, medical domains, or customer chatbots**—common areas where it's especially dangerous.

You said:

What is fine tuning

ChatGPT said:

What is Fine-Tuning in AI?

Fine-tuning is the process of taking a **pre-trained model** (like GPT, BERT, or Stable Diffusion) and continuing its training on a **smaller, task-specific dataset**. This allows the model to **specialize** in a particular domain or task while leveraging knowledge it already learned from large-scale pretraining.

Analogy

Think of pretraining as getting a general education (like a high school diploma) and fine-tuning as specialized training for a specific job (like learning to be a lawyer or a data scientist).

How Fine-Tuning Works

1. **Start with a pre-trained base model** (e.g., GPT-3, BERT, CLIP).
2. **Prepare a labeled dataset** related to your task (e.g., legal Q&A, medical instructions, customer support).
3. **Train** the model further on this dataset using **supervised learning**.
4. The model adjusts its weights slightly to better fit the new task.

Types of Fine-Tuning

Type	Description	Example
Full fine-tuning	Update all model parameters	High cost, high accuracy
Parameter-efficient fine-tuning	Update only a subset (e.g., LoRA, adapters)	Faster and cheaper
Instruction tuning	Train on task instructions and outputs	Improves following human prompts
Domain adaptation	Fine-tune on domain-specific data	Legal, medical, finance

Fine-Tuning vs Prompt Engineering

Feature	Fine-Tuning	Prompt Engineering
Needs training?	✅ Yes	❌ No
Custom behavior	✅ Strong	⚠️ Limited
Cost	💰 High (compute, data)	● Low
Flexibility	🎯 High	⚠️ May require retries

Example

You take GPT-3 and fine-tune it on **legal documents** with question-answer pairs. Now the model:

- Understands legal jargon
- Answers legal queries more reliably
- Reduces hallucinations in that domain

Considerations

- Needs **clean, labeled data**
- Can cause **catastrophic forgetting** (losing general knowledge)
- Needs **GPU resources**
- Might introduce **bias** if the fine-tuning data is skewed

Let me know if you'd like a step-by-step guide or code example to fine-tune a model using Hugging Face or OpenAI APIs.

You said:

explain fine tuning in depth

ChatGPT said:

Absolutely. Let's dive **in-depth** into **fine-tuning** — a foundational concept in building specialized AI systems like domain-specific chatbots, assistants, and image generators.

What is Fine-Tuning?

Fine-tuning is a process where a **pre-trained model** (like GPT, BERT, or a vision transformer) is **further trained** on a **smaller, task-specific dataset**. It enables the model to adapt to a new domain or task **without training from scratch**, saving time and compute.

Why Fine-Tuning Is Needed

Pre-trained models are trained on **broad, general-purpose datasets** (e.g., Common Crawl, Wikipedia), but:

- They might not **understand specific domain jargon** (like medical/legal terms).
- They may **hallucinate or underperform** on specialized tasks (e.g., summarizing financial reports).
- Custom tasks like **chatbots, classification, summarization, or code generation** often need **tailored behavior**.

Fine-tuning fixes this by **specializing** the model for your task.

How Fine-Tuning Works

1. Start with a Pre-Trained Model

Examples:

- GPT (OpenAI), LLaMA, Falcon — for text generation

- BERT, RoBERTa — for classification and embeddings
- CLIP, Stable Diffusion — for image generation and understanding

These models already "know" a lot about the world.

2. Prepare a Custom Dataset

- Format: Usually in **input-output pairs**, like:

```
json
CopyEdit
{
  "prompt": "Translate English to French: 'Hello'",
  "completion": "Bonjour"
}
```

- File types: JSONL, CSV, or instruction-style format
- Types of data:
 - Q&A pairs
 - Dialogue turns
 - Instruction-response examples
 - Code snippets
 - Domain documents

3. Select the Fine-Tuning Method

Method	Description	Use case
Full fine-tuning	Update all parameters of the model	Best when lots of data and compute are available
LoRA (Low-Rank Adaptation)	Injects small trainable layers	Memory-efficient; great for LLMs
Adapters	Small modules inserted into each transformer layer	Modular and reusable

Prefix Tuning / Prompt Tuning

Learns a trainable prompt vector

Small footprint; good for inference-time flexibility

4. Training Setup

Key considerations:

- **Learning rate:** Start small (e.g., $1e-5$ or $2e-5$)
- **Epochs:** 3–10 depending on overfitting risk
- **Loss function:** Usually Cross-Entropy Loss for language models
- **Batch size:** Limited by GPU memory
- **Tokenizer:** Use the tokenizer corresponding to the base model (e.g., GPT-2's tokenizer for GPT-2)

5. Evaluate the Fine-Tuned Model

Use evaluation metrics based on the task:

- **Text classification:** Accuracy, F1
- **Text generation:** BLEU, ROUGE, METEOR
- **Q&A:** EM (Exact Match), F1
- **Embeddings/Retrieval:** Cosine similarity, NDCG

Also use **manual review** to check for:

- Hallucinations
- Tone and coherence
- Bias or unsafe content



Example: Fine-Tuning GPT-2 on Custom Instructions

python
CopyEdit

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel, Trainer,
TrainingArguments, TextDataset, DataCollatorForLanguageModeling

# Load base model and tokenizer
model = GPT2LMHeadModel.from_pretrained("gpt2")
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

# Load and tokenize dataset
train_dataset = TextDataset(
    tokenizer=tokenizer,
    file_path="data/custom_train.txt",
    block_size=128
)

# Prepare data collator
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer, mlm=False
)

# Training configuration
training_args = TrainingArguments(
    output_dir="./gpt2-finetuned",
    overwrite_output_dir=True,
    num_train_epochs=3,
    per_device_train_batch_size=4,
    save_steps=500,
    save_total_limit=2,
)

# Train
trainer = Trainer(
    model=model,
    args=training_args,
    data_collator=data_collator,
    train_dataset=train_dataset,
)

trainer.train()
```

Trade-offs of Fine-Tuning

Pros	Cons
Tailored performance	Can overfit on small datasets
Reduces hallucination	High computational cost
Enables new tasks	Needs good quality data
Boosts domain alignment	May forget general knowledge (catastrophic forgetting)

Alternatives to Fine-Tuning

Technique	Description
Prompt Engineering	No training, just smart prompt design
RAG	Retrieve facts to ground LLM output
Tool Augmentation	Use LLMs with tools like calculators, databases
Embedding-based retrieval	Use LLMs for semantic search instead of generation

Summary

- **Fine-tuning** adapts a pre-trained model to your specific task or domain.
- It can be **full** or **parameter-efficient** (LoRA, adapters).
- It needs **high-quality data**, **compute resources**, and **evaluation methods**.
- It enables more **accurate**, **context-aware**, and **safe** AI systems.

Would you like a real fine-tuning case study (e.g., fine-tuning GPT for legal advice or code generation)?

You said:

how to solve the issue of rag application giving incorrect answer or unable to answer

ChatGPT said:

Great question. A **RAG (Retrieval-Augmented Generation)** system can give **incorrect** or **no answers** due to multiple reasons — poor retrieval, embedding mismatch, hallucination, or context truncation. Here's a structured way to **diagnose and fix** this.

✓ Step-by-Step Guide to Solving RAG Output Issues

🧩 1. Problem: Retrieved Documents Are Irrelevant

🔍 Cause:

- Bad or low-quality embeddings
- Poor chunking strategy
- Incorrect similarity metric

✓ Fix:

- Use a **better embedding model** (e.g., all-MiniLM, bge-large-en, or OpenAI's text-embedding-3-small)
- Tune chunk size (e.g., 200–500 words with overlap)
- Use **hybrid search** (BM25 + vector search) instead of vector-only
- Evaluate using **Top-K document precision**

🧠 2. Problem: LLM Hallucinates Despite Correct Context

🔍 Cause:

- LLM not grounded in retrieved docs
- Prompting doesn't restrict model to use context
- Too many unrelated docs included



- Use **better prompts**, like:

“Answer the question only using the following documents. If you don’t know, say you don’t know.”

- Use **context filtering or reranking** with tools like:
 - Cohere Rerank
 - SentenceTransformer + cosine similarity rerank
- Reduce number of context documents passed to the LLM

3. Problem: LLM Doesn’t Answer at All (or Says 'I don’t know')



- Relevant document not retrieved
- Context cutoff due to long input
- Retrieval failure



- Increase the top_k in vector search (e.g., from 3 to 10)
- Compress context (e.g., summarize retrieved docs first)
- Use **multi-stage retrieval** (coarse search → fine reranking)
- Implement **fallbacks**, e.g.:

text

CopyEdit

If answer is not found, ask follow-up question or return “I don’t know”.

4. Problem: Context Window Exceeded

Cause:

- Too many long documents retrieved
- Truncation happens mid-document

Fix:

- Chunk smaller (e.g., 300 tokens + 50 overlap)
- Use **token-aware truncation** (e.g., with tiktoken)
- Prioritize documents using a scoring mechanism (relevance > length)

5. Problem: Knowledge Base Is Incomplete or Outdated

Cause:

- You haven't ingested the right documents
- Or vector DB isn't refreshed after updates

Fix:

- Ensure **relevant documents are ingested**
- Re-index after any KB update
- Schedule periodic refreshes of the embedding store

Bonus: Iterative RAG (Self-Ask, ReAct, etc.)

If you still get wrong answers, try **multi-step RAG**:

- Break down queries
- Retrieve-per-step (not once)
- Use **agents** like LangChain's ReAct, or Self-Ask with Search

Diagnostic Checklist

Check	Tool or Method
Are the top-k retrieved docs relevant?	Manually inspect or use retriever recall
Are the embeddings semantic enough?	Try different models (e.g., OpenAI, Cohere, bge)
Is the prompt grounding the model?	Test with constrained prompts
Is the context too long?	Count tokens (use tiktoken or transformers tokenizer)
Are retrieval and generation decoupled?	Log separately to debug

Prompt Engineering Template

text

CopyEdit

Answer the following question using only the information provided in the context.

If the answer is not present in the context, reply with "I don't know."

Context:

{retrieved_docs}

Question:

{user_query}

Advanced Fix: Use Feedback Loops

- Log every query + retrieved docs + LLM output

- Use human or LLM evaluation to tag good/bad results
- Fine-tune reranker or retriever with this feedback

Would you like a **RAG debugging template** or a **real-world example** where a RAG system was failing and improved using these methods?

You said:

yes

ChatGPT said:

Great! Here's a **real-world RAG debugging template and example**, showing how a failing RAG system was diagnosed and improved — step-by-step.

RAG Debugging & Improvement Template

This template helps log, evaluate, and iteratively improve RAG-based apps:

1. Log These for Each Query

Item	Description
Query	User input
Retrieved Docs	Top-K document snippets (with scores)
LLM Prompt	Final constructed prompt
LLM Output	Model's answer
Ground Truth (Optional)	Human-labeled correct answer
Evaluation Score	Manual ( / ), BLEU/F1, or LLM-based rating

2. Debugging Checklist

Question

Fix If "No"

Are top-K docs semantically relevant?

Is prompt instructing model clearly?

Are docs being truncated?

Is model hallucinating despite relevant docs?

Are correct docs missing?

Change embedding model or chunking

Use constrained prompts

Reduce context size or use reranker

Apply reranking or stricter prompts

Improve data ingestion, search strategy

Real-World Example: Legal Q&A Chatbot (RAG)

Problem:

A legal chatbot built using OpenAI + Pinecone RAG was giving **inaccurate answers** about corporate law in India.

Case Before Fixes

Step	Detail
User Query	"Can an LLP raise venture capital in India?"
Retrieved Docs	Mentioned "LLP taxation," "LLP registration process"
LLM Output	"Yes, LLPs can raise venture capital freely in India."
Issue	Hallucination — LLPs actually face legal restrictions on equity financing
Why?	Retrieval failed to get the relevant regulatory clause ; prompt was too generic

Fixes Applied

1. Better Embedding Model

Switched from text-embedding-ada-002 → bge-large-en-v1.5 for better semantic matching of legal terms.

✓ 2. Improved Chunking

- Old: 1,000-word chunks (too broad)
- New: 400-token chunks with 50-token overlap
- Legal sections now precisely retrievable

✓ 3. Prompt Refinement

text

CopyEdit

Use the context below to answer the question. If you don't know the answer, say "I don't know." Do not make up facts.

Context:

{retrieved_docs}

Question:

{user_query}

✓ 4. Hybrid Search

- Combined **BM25 keyword search + vector similarity**
- Ensured regulatory keywords like "equity," "capital," "investment" were not missed

✓ 5. Reranking

Used Cohere Rerank to select top-3 most relevant chunks from top-10 retrieved.

✓ After Fixes

Step

Detail

User

Quer "Can an LLP raise venture capital in India?"

y

Retri

eved Now include MCA guidelines, Companies Act comparison

Docs

LLM “LLPs cannot issue equity shares and thus cannot raise venture capital in the

Outp traditional sense. They may receive partner contributions or convert to a

ut company structure.”

Resu

lt

✓ Factually correct, grounded, and safe response

Sample Logging JSON Format (for feedback loop)

json

CopyEdit

```
{
  "query": "Can an LLP raise venture capital in India?",
  "retrieved_docs": ["LLPs are not allowed to issue equity shares...",
"...must convert to Pvt Ltd"],
  "llm_prompt": "Use the context below to answer the question...",
  "llm_response": "LLPs cannot issue equity shares...",
  "human_label": "correct",
  "issues_fixed": ["embedding", "chunking", "prompt", "reranking"]
}
```

Tools You Can Use

Tool	Purpose
LangChain or LlamaIndex	Easy RAG pipelines
Pinecone, Weaviate, FAISS	Vector DB
Cohere Rerank / BGE-Reranker	Reranking
LLM-as-a-Judge	Use GPT-4 to auto-score answers
Weights & Biases / PromptLayer	Logging + analytics

Summary

Improving RAG systems involves:

- Better **retrieval** (embedding, chunking, hybrid search)
- Smarter **prompting**
- Strict **grounding policies**
- Evaluation and **feedback loops**

Would you like me to generate a **ready-made notebook** or code template for building and debugging a RAG system?

You said:

dimensionality curse in ai and how to resolve it

ChatGPT said:

The **Curse of Dimensionality** is a common problem in AI and machine learning, especially when dealing with high-dimensional data (lots of features). Here's a deep breakdown:

What is the Curse of Dimensionality?

The **curse of dimensionality** refers to various problems that arise when the number of input features (dimensions) becomes very high. As dimensionality increases:

1. **Data becomes sparse.**
2. **Distance metrics become less meaningful.**
3. **Model complexity grows**, leading to overfitting.
4. **Computation becomes expensive.**

Why It's a Problem in AI/ML?

Problem	Description
Sparsity	In high-dimensional space, data points are far apart. It becomes harder to find meaningful clusters or nearest neighbors.
Overfitting	More dimensions = more ways to fit noise, not the signal. Models memorize instead of generalizing.
Distance Metrics Fail	In high-D, all distances converge (e.g., Euclidean distance loses its effectiveness).
More Samples Needed	To maintain the same coverage, the required number of training samples increases exponentially with dimensions.

Example

- Suppose you have 10 data points in 1D: you can cover the space well.
- But if each point has 1000 features (dimensions), those 10 points **barely cover** anything.
- Now distance-based models like **KNN** or **clustering** perform poorly.

How to Solve or Reduce It

1. Dimensionality Reduction

Technique	Purpose	Example Tools
PCA (Principal Component Analysis)	Linear transformation to reduce dimensions while preserving variance	<code>sklearn.decomposition.PCA</code>
t-SNE / UMAP	Non-linear techniques for visualization or clustering	<code>sklearn.manifold.TSNE</code> , <code>umap-learn</code>
Autoencoders	Neural networks that compress and decompress data	TensorFlow / PyTorch

Feature selection

Keep only important features based on correlation, mutual information, etc.

SelectKBest, RFE

2. Use Regularization

- Models like **Ridge (L2)** or **Lasso (L1)** penalize large weights, reducing overfitting in high-D.
- Helps control model complexity even when the feature space is large.

3. Use Tree-Based Models

- Models like **Random Forests** and **Gradient Boosting Trees (XGBoost, LightGBM)** handle high-dimensional data better than distance-based models.
- They perform **implicit feature selection** during training.

4. Embedded Techniques

- **L1 Regularization (Lasso)** automatically sets some weights to zero — acts like feature selection.
- Used in **Linear models**, **SVMs**, and **Logistic Regression**.

5. Feature Engineering

- Remove redundant or irrelevant features using domain knowledge.
- Combine multiple features into meaningful aggregates.

Real-World Example

Problem:

A credit scoring system with 1,000 features was **overfitting** and producing unstable predictions.

Fixes Applied:

- Ran PCA → reduced to top 50 components explaining 95% variance.
- Used L1-penalized logistic regression → auto-pruned features.
- Switched from KNN → Random Forest (better for high-D).
- Model became more stable and generalizable.

Summary

Concept	Fix
Sparsity	Dimensionality reduction (PCA, Autoencoders)
Distance fails	Switch from KNN to tree-based models
Overfitting	Regularization (L1/L2), Feature selection
Computation	Use only most relevant features